

Question 33 — Object-Oriented Analysis & SDLC Models

Reg. No. 192571012

PART (a): Online Banking System

System Overview

A secure online banking platform allows customers to manage accounts, transfer funds, pay bills, and apply for loans while supporting concurrent users, high-volume transactions, and strict regulatory security requirements.

1. Key Classes, Attributes and Methods

Class	Key Attributes	Key Methods
User	userID, name, email, password (hashed)	login(), logout(), changePassword(), getProfile()
Customer (extends User)	customerID, address, phoneNumber, kycStatus	applyForLoan(), viewTransactionHistory()
Account	accountNumber, accountType, balance, status, openDate	deposit(), withdraw(), getBalance(), freeze()
Transaction	transactionID, amount, timestamp, type, status	execute(), rollback(), validate(), getReceipt()
FundTransfer (extends Transaction)	senderAccount, receiverAccount, ifscCode	initiateTransfer(), verifyBeneficiary()
BillPayment (extends Transaction)	billID, billAmount, dueDate	payBill(), scheduleBill()
LoanApplication	loanID, loanType, amount, tenure, status	apply(), approve(), reject(), calculateEMI()
Bank	bankID, name, regulationCode	openAccount(), closeAccount(), auditLog()
SecurityModule	sessionToken, otpSecret, failedAttempts	generateOTP(), verifyOTP(), detectFraud(), lockAccount()

2. Relationships Between Classes

Inheritance (IS-A): Customer extends User. FundTransfer and BillPayment extend Transaction — modelling specialised transaction behaviours.

Association: Customer has one or more Accounts (1-to-many). Bank manages multiple Customers and Accounts.

Aggregation: Bank aggregates Accounts. If the Bank is reorganised, Accounts can be migrated independently.

Composition: Account is composed of Transactions. Transactions cannot exist without a parent Account.

Dependency: Transaction depends on SecurityModule for OTP verification and fraud detection.

Realisation: LoanApplication implements an Approvable interface with approve(), reject(), and review().

3. Polymorphism

The **execute()** method is defined in the base Transaction class and overridden in each subclass:

- FundTransfer.execute() debits the sender and credits the receiver atomically.
- BillPayment.execute() contacts the biller API and marks the bill as paid.
- LoanDisbursement.execute() credits the loan amount to the customer account.

Similarly, Notification.send() is overridden by EmailNotification, SMSNotification, and PushNotification — achieving runtime polymorphism.

4. Recommended SDLC Model — Spiral Model

Recommended Model: Spiral Model (risk-driven iterative development).

Justification:

- Risk Management:** Banking carries high financial and regulatory risk. Each Spiral iteration includes an explicit risk-analysis phase to identify security vulnerabilities and compliance gaps early.
- Incremental Delivery:** Core features (login, account view) are delivered in early spirals; transactions, bill payments, and loans follow in subsequent iterations.
- Security Audits Per Spiral:** Each iteration ends with a security review and penetration testing — crucial for a financial system.
- Handles Evolving Regulations:** Banking regulations (RBI, PCI-DSS, GDPR) evolve. The iterative nature accommodates changes without a full re-design.
- Prototype Validation:** Early-stage prototypes validate UI/UX and transaction workflows with stakeholders before full implementation.

Why not Waterfall? Waterfall assumes complete, stable requirements upfront — impossible for a system with evolving security regulations and complex concurrent transaction logic.

PART (b): Smart Library Management System

System Overview

A university library automates book cataloguing, member registration, book lending, return tracking, and overdue fine calculation — supporting both physical and digital book management.

1. Key Classes, Attributes and Methods

Class	Key Attributes	Key Methods
Person (abstract)	personID, name, email, phone	register(), getDetails()
Member (extends Person)	memberID, membershipType, borrowedBooks, fineAmount	borrowBook(), returnBook(), payFine()
Librarian (extends Person)	staffID, designation	addBook(), removeBook(), approveLoan()
Book (abstract)	bookID, title, author, ISBN, genre, year	getDetails(), search()
PhysicalBook (extends Book)	shelfLocation, copies, availableCopies	checkAvailability(), reserve()
DigitalBook (extends Book)	fileFormat, fileSize, downloadURL, license	download(), preview(), checkLicense()
LoanRecord	loanID, member, book, issueDate, dueDate	issueBook(), returnBook(), calculateFine()
Catalogue	catalogueID, books[]	addBook(), removeBook(), searchByTitle(), searchByAuthor()
Fine	fineID, amount, paidStatus, dueDate	calculate(), pay(), waive()

2. Relationships

- Inheritance (IS-A):** Member and Librarian extend abstract Person. PhysicalBook and DigitalBook extend abstract Book.
- Association:** Member borrows Book through a LoanRecord (many-to-many resolved via LoanRecord association class). Librarian manages Catalogue.

Aggregation (weak HAS-A): Catalogue aggregates many Books — Books exist independently and can belong to multiple catalogues.

Composition (strong HAS-A): LoanRecord is composed of a Fine — Fine cannot logically exist without an associated LoanRecord.

Dependency: LoanRecord depends on Notification to send due-date and overdue reminders.

3. Polymorphism in the Library System

The abstract Book class enables uniform handling of Physical and Digital books:

- Book.borrow(member): PhysicalBook reduces availableCopies; DigitalBook checks active license count.
- Book.returnBook(member): PhysicalBook increments availableCopies; DigitalBook revokes the download session.
- Book.search(query): PhysicalBook searches by shelf location; DigitalBook searches metadata and full-text index.
- Person.register(): Member registration creates a library card; Librarian registration assigns staff ID and permissions.

This enables generic code like: for book in catalogue.books: book.borrow(member) — working seamlessly for both book types.

4. Recommended SDLC Model — Waterfall Model

Recommended Model: Waterfall Model.

Justification:

Stable, Well-Defined Requirements: University library operations (cataloguing, lending, fines) are clearly understood with minimal ambiguity — ideal for Waterfall's sequential phases.

Documentation-Driven: Academic institutions require thorough documentation for audits and compliance. Waterfall produces complete SRS, design documents, and test plans at each phase.

Low Risk of Requirement Change: Library operations follow well-established processes that rarely change drastically, making upfront complete planning feasible.

Budget and Timeline Predictability: Waterfall allows accurate estimation of time and cost — important for university IT budget cycles.

Clear Phase Completion: Each phase (Requirements, Design, Coding, Testing, Deployment) has defined deliverables and sign-off — matching institutional governance.

Why not Agile? Agile suits projects with unclear or rapidly-changing requirements. A library system with predefined, stable processes does not benefit from sprint-based iterations.

Waterfall Phases for Smart Library System

Phase	Activities
1. Requirements	Gather library rules, lending policies, fine structures, and digital access rights from librarians and administrators.
2. System Design	Design class diagrams, ER diagrams, database schema, and UI wireframes for web/mobile interface.
3. Implementation	Develop modules: Catalogue, Member Management, Loan Processing, Fine Calculation, Digital Access Control.
4. Testing	Unit testing per module, integration testing of borrow/return workflows, UAT with library staff.
5. Deployment	Deploy on university servers, train library staff, migrate existing catalogue data.
6. Maintenance	Bug fixes, software updates, adding new digital formats or third-party API integrations.

Summary Comparison

Aspect	Online Banking System (a)	Smart Library System (b)
SDLC Model	Spiral Model	Waterfall Model
Reason for Model	High risk, evolving regulations, security-critical	Stable, well-defined institutional requirements
Key Polymorphism	Transaction.execute() overridden by subclasses	Book.borrow() / returnBook() overridden by subclasses
Main Relationship	Composition: Account contains Transactions	Aggregation: Catalogue aggregates Books
Primary Concern	Security, concurrency, regulatory compliance	Accuracy, cataloguing, fine management